

The JOKER Protocol: A High-Level Overview

Opportunistic Routing for Ad-Hoc Networks

Project Presentation Document

Contents

1	Introduction: What is the JOKER Protocol?	2
2	Why Are We Using It?	2
3	Where Does It Live? (Network Layer Perspective)	2
4	Why is JOKER Highly Energy Efficient?	2
5	How Are We Implementing It?	3
6	How Will We Demo It?	3
7	Summary for the Class	3

1 Introduction: What is the JOKER Protocol?

JOKER (auto-adJustable Opportunistic acKnowledgment/timEr-based Routing) is a novel **opportunistic routing protocol** designed for Mobile Ad-hoc Networks (MANETs).

Unlike traditional routing protocols that calculate a single, fixed path between a sender and a receiver, an opportunistic protocol takes advantage of the wireless medium's "broadcast" nature. When a node transmits data, multiple neighboring nodes (called **candidates**) can overhear it. JOKER smartly coordinates these neighbors to decide which one is in the best position to forward the packet next. This ensures that even if nodes are moving or links are dropping, the data always finds the fastest and most reliable path to its destination.

2 Why Are We Using It?

We are using JOKER primarily for two critical reasons:

- **Optimized for Multimedia (High QoE):** Modern networks are heavy with video streaming and multimedia. JOKER is specifically designed to provide high Quality of Experience (QoE) for heavy traffic, reducing delays and packet loss compared to older protocols.
- **Energy Efficiency:** Mobile devices rely on batteries. Routing operations usually drain a lot of power. JOKER minimizes unnecessary network transmissions and control overhead, allowing device wireless cards to sleep more often and save significant battery life.

3 Where Does It Live? (Network Layer Perspective)

If we look at the standard OSI networking model, **JOKER operates exactly between the Link (MAC) layer and the Network layer.**

This means it sits right below standard IP routing. When an application sends data (like IPv4, TCP, or UDP), JOKER intercepts it, wraps it in a special "JOKER Header," and hands it directly to the WiFi card (the Link layer).

Why is this beneficial? Because it means we do not need to modify the underlying Android Operating System's WiFi drivers or IP stack. JOKER works by simply putting the wireless card into "promiscuous mode" (overhearing all local traffic) and handling the routing logic entirely in software.

4 Why is JOKER Highly Energy Efficient?

JOKER achieves its remarkable energy efficiency through three main mechanisms:

1. **Dynamic Control Messages:** Most protocols spam the network with "Hello" messages every second to discover neighbors. JOKER dynamically adjusts this interval based on current network traffic. If there is no traffic, it slows down the messages, saving power.

2. **Distance-Progress Routing:** Instead of picking the nearest, safest neighbor (which requires many small hops and lots of transmissions), JOKER uses a metric that rewards nodes that are physically further away. Longer hops mean fewer total transmissions to reach the destination.
3. **Smart Coordination:** To prevent multiple candidates from forwarding the exact same packet, JOKER uses an elegant *Timer-based* or *ACK-based* coordination scheme. The best candidate forwards immediately, and the others hear it and cancel their own transmissions, preventing duplicate network spam.

5 How Are We Implementing It?

Our implementation consists of a dual-layer architecture:

- **The C++ Core:** The heavy lifting, routing logic, and protocol algorithms described in the paper are implemented in highly efficient native C++.
- **The Android App (JNI Bridge):** We built a modern Kotlin/Jetpack Compose Android application that serves as the user interface. Using the Java Native Interface (JNI) via the Android NDK and CMake, our Android app hooks directly into the C++ core. This allows the Android phone to execute the complex protocol at metal-level speeds while providing a clean, accessible UI for the user.

6 How Will We Demo It?

For our class presentation, the demo will be interactive and visual:

1. We will load the built `app-debug.apk` onto multiple Android phones.
2. During the demo, we will open the app and tap the **"Start Protocol"** button. This will instantly initialize the C++ JOKER core via our JNI bridge.
3. The devices will begin communicating opportunistically. We will show the live logs on the screen, proving that the native C++ protocol is actively running and managing the network mesh directly from the Android devices.

7 Summary for the Class

When explaining this to the class, keep the focus on the big picture: **JOKER is a modern, energy-saving mesh protocol that intelligently uses multiple neighbors to forward data, making it perfect for streaming video across mobile phones without killing their batteries.**